

Preentrega Backend con Java

Sistema Control de Inventario - TechLab

José Miguel Paz Portilla

6 de octubre de 2026

Índice

1. Introducción y Arquitectura del Sistema	3
1.1. Distribución Modular de Responsabilidades	3
1.2. Patrón de Inyección de Dependencias	3
2. Creación del proyecto	3
2.1. build_new_project.sh	3
3. Ejecución del proyecto	7
3.1. run_project.sh	7
4. Código	8
4.1. Main.java	8
4.2. Validador.java	11
4.3. Producto.java	15
4.4. ProductoService.java	18
4.5. ProductoNoEncontradoException.java	22
4.6. StockInsuficienteException.java	23
4.7. MenuProducto.java	24
5. Corridas	28
6. Pruebas Unitarias	30
6.1. Estrategia de Testing y Robustez del Dominio (Agregado)	31
6.2. ProductoServiceTest.java	31
7. Conclusiones	35

1. Introducción y Arquitectura del Sistema

El presente proyecto documenta la preentrega de la plataforma de control de inventario de TechLab. El software se modeló utilizando el paradigma de Programación Orientada a Objetos (POO) bajo una arquitectura desacoplada en capas. Este enfoque asegura un diseño de software mantenible y modular, aislando la lógica transaccional de los mecanismos de presentación.

1.1. Distribución Modular de Responsabilidades

La separación de paquetes encarna el principio de Cohesión y Responsabilidad Única (SRP):

- **model:** Contiene las entidades puras del negocio (POJOs), responsables de modelar el estado del objeto.
- **service:** Capa lógica central encargada de las transacciones CRUD y la manipulación de la persistencia volátil en memoria.
- **exception:** Centralización funcional de los errores de negocio mapeados mediante excepciones personalizadas.
- **ui:** Gestión del flujo de consola y entrada/salida interactiva de datos.
- **util:** Módulos utilitarios estáticos para el procesamiento genérico y la sanitización de entradas por teclado.

1.2. Patrón de Inyección de Dependencias

Para evitar el acoplamiento duro, se implementó el patrón de Inyección de Dependencias por Constructor dentro de la clase `MenuProducto`. Al recibir el `Scanner` y la instancia de `ProductoService` desde el exterior, la interfaz queda desacoplada de la inicialización de sus componentes. Esta práctica es la base técnica fundamental para la futura migración hacia frameworks empresariales como Spring Boot.

2. Creación del proyecto

El proyecto por problemas de velocidad del ordenador se realizó desde la terminal de linux. Para crear un nuevo proyecto java desde la terminal se creó un script `build_new_project.sh` el cual crea las carpeta como lo necesita maven, crea el archivo `pom.xml`, crea las carpetas `ui`, `util`, `model`, `exception` y `service`. Tambien crea un `Main.java` con un Hola mundo de prueba.

2.1. build_new_project.sh

Listing 1: Script de automatización `build_new_project.sh`

```
1 #!/bin/bash
2
3 # Detener el script inmediatamente si ocurre algun error
4 set -e
5
```

```

6 # =====
7 # VALIDACION DE ARGUMENTOS DE LINEA DE COMANDO
8 # =====
9 # Si el primer argumento ($1) esta vacio, muestra un error
  explicativo y sale.
10 if [ -z "$1" ]; then
11     echo "Error: No especificaste el nombre del proyecto."
12     echo "Uso correcto: $0 <nombre_del_proyecto>"
13     echo "Ejemplo: $0 preentrega"
14     exit 1
15 fi
16
17 # Asignamos el primer argumento de la terminal a nuestra variable
18 NUM_EJERCICIO="$1"
19
20 # Definicion de rutas estandar de Maven usando la variable
21 PACKAGE_DIR="src/main/java/com/josepaz/$NUM_EJERCICIO"
22 TEST_DIR="src/test/java/com/josepaz/$NUM_EJERCICIO"
23 RESOURCES_DIR="src/main/resources"
24
25 echo "=== 1. Generando estructura de carpetas estandar de Maven y
  paquetes internos para: $NUM_EJERCICIO ==="
26 # Creamos la ruta base del paquete y del entorno de pruebas
27 mkdir -p "$PACKAGE_DIR"
28 mkdir -p "$TEST_DIR"
29 mkdir -p "$RESOURCES_DIR"
30
31 # Creamos las subcarpetas de arquitectura dentro de la carpeta del
  ejercicio
32 mkdir -p "$PACKAGE_DIR/exception"
33 mkdir -p "$PACKAGE_DIR/model"
34 mkdir -p "$PACKAGE_DIR/service"
35 mkdir -p "$PACKAGE_DIR/util"
36 mkdir -p "$PACKAGE_DIR/ui"
37
38 echo "=== 2. Verificando archivo Java principal ==="
39 # Solo crea el Main.java de ejemplo inicial si el archivo NO existe
  .
40 if [ ! -f "$PACKAGE_DIR/Main.java" ]; then
41     echo "Creando archivo Java de ejemplo (Main.java)..."
42     cat << EOF > "$PACKAGE_DIR/Main.java"
43 package com.josepaz.$NUM_EJERCICIO;
44
45 public class Main
46 {
47     public static void main(String[] args)
48     {
49         System.out.println("Hola Mundo desde Main.java en Maven (
          $NUM_EJERCICIO)");
50     }
51 }
52 EOF
53 else

```

```

54     echo "El archivo Main.java ya existe. Respetando tu codigo
        actual."
55 fi
56
57 echo "=== 3. Generando archivo pom.xml con soporte JUnit 5 y
    Surefire ==="
58 # Se elimino la barra invertida de la etiqueta mainClass para que
    Bash inyecte el valor real
59 cat << EOF > pom.xml
60 <?xml version="1.0" encoding="UTF-8"?>
61 <project xmlns="http://apache.org"
62         xmlns:xsi="http://w3.org"
63         xsi:schemaLocation="http://apache.org http://apache.org">
64     <modelVersion>4.0.0</modelVersion>
65
66     <groupId>com.josepaz</groupId>
67     <artifactId>hola-mundo-terminal</artifactId>
68     <version>1.0-SNAPSHOT</version>
69
70     <properties>
71         <maven.compiler.source>21</maven.compiler.source>
72         <maven.compiler.target>21</maven.compiler.target>
73         <project.build.sourceEncoding>UTF-8</project.build.
            sourceEncoding>
74     </properties>
75
76     <dependencies>
77         <!-- Dependencia para habilitar las pruebas unitarias de
            JUnit 5 -->
78         <dependency>
79             <groupId>org.junit.jupiter</groupId>
80             <artifactId>junit-jupiter</artifactId>
81             <version>5.10.2</version>
82             <scope>test</scope>
83         </dependency>
84     </dependencies>
85
86     <build>
87         <plugins>
88             <!-- Plugin para compilar con tu version Java 21 -->
89             <plugin>
90                 <groupId>org.apache.maven.plugins</groupId>
91                 <artifactId>maven-compiler-plugin</artifactId>
92                 <version>3.13.0</version>
93                 <configuration>
94                     <source>21</source>
95                     <target>21</target>
96                 </configuration>
97             </plugin>
98
99             <!-- Plugin Surefire moderno para mapear y listar la
                ejecucion de JUnit 5 -->
100            <plugin>

```

```

101         <groupId>org.apache.maven.plugins</groupId>
102         <artifactId>maven-surefire-plugin</artifactId>
103         <version>3.2.5</version>
104     </plugin>
105
106     <!-- Plugin Exec apuntando a tu clase dinamica com.
107         josepaz -->
108     <plugin>
109         <groupId>org.codehaus.mojo</groupId>
110         <artifactId>exec-maven-plugin</artifactId>
111         <version>3.5.0</version>
112         <configuration>
113             <mainClass>com.josepaz.${NUM_EJERCICIO}.Main</
114                 mainClass>
115             </configuration>
116         </plugin>
117     </plugins>
118 </build>
119 </project>
120 EOF
121
122 echo "=== 4. Limpiando y compilando el proyecto ==="
123 mvn clean compile
124
125 echo "=== 5. Ejecutando la clase Main de $NUM_EJERCICIO ==="
126 mvn exec:java
127
128 echo "=== 6. Ejecutando las pruebas unitarias con JUnit 5 ==="
129 mvn test

```

Este script debe tener permiso de ejecución que se otorga del siguiente modo:
chmod +x build_new_proyect.sh

El la Figura 1 se muestra como crear el proyecto de la preentrega.

```

josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$ ./build_new_project.sh preentrega
=== 1. Generando estructura de carpetas estándar de Maven y paquetes internos para: preentrega ===
=== 2. Verificando archivo Java principal ===
Creando archivo Java de ejemplo (Main.java)...
=== 3. Generando archivo pom.xml ===
=== 4. Limpiando y compilando el proyecto ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ hola-mundo-terminal ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug target 21] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.502 s
[INFO] Finished at: 2026-10-01T00:59:38-03:00
[INFO]
=== 5. Ejecutando la clase Main de preentrega ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.5.0:java (default-cli) @ hola-mundo-terminal ---
¡Hola Mundo desde Main.java en Maven (preentrega)!
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.874 s
[INFO] Finished at: 2026-10-01T00:59:42-03:00
[INFO]
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$

```

Figura 1: Creación de un nuevo proyecto java

3. Ejecución del proyecto

3.1. run_project.sh

Listing 2: Script de ejecucion run_project.sh

```

1  #!/bin/bash
2
3  # Detener el script inmediatamente si ocurre algun error
4  set -e
5
6  # Verificamos si pasaste el argumento por la terminal
7  if [ -z "$1" ]; then
8      echo "Error: No especificaste el proyecto a ejecutar."
9      echo "Uso correcto: $0 <nombre_del_proyecto>"
10     echo "Ejemplo: $0 preentrega"
11     exit 1
12 fi
13
14 # Asignamos el argumento a una variable clara
15 EJERCICIO="$1"
16

```

```

17 echo "=== Ejecutando la aplicacion con Maven ($EJERCICIO) ==="
18 mvn package exec:java -Dexec.mainClass="com.josepaz.${EJERCICIO}.
    Main"

```

En la Figura 2 se muestra como ejecutar el proyecto de la preentrega.

```

josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$ ./run_project.sh preentrega
=== Ejecutando la aplicacion con Maven (preentrega) ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 7 source files with javac [debug target 21] to target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/josepaz/JAVA/java-preentrega/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed dependency.
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hola-mundo-terminal ---
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ hola-mundo-terminal ---
[INFO] Building jar: /home/josepaz/JAVA/java-preentrega/target/hola-mundo-terminal-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- exec-maven-plugin:3.5.0:java (default-cli) @ hola-mundo-terminal ---
Se cargaron 5 productos de prueba.

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:

```

Figura 2: Ejecución del proyecto java

4. Código

4.1. Main.java

Listing 3: Clase Main.java (Pre-entrega)

```

1 package com.josepaz.preentrega;
2
3 import java.util.Scanner;
4
5 import exception.ProductoNoEncontradoException;
6 import exception.StockInsuficienteException;
7 import model.Producto;
8 import service.ProductoService;
9 import ui.MenuProducto;
10 import util.Validador;
11

```

```

12  /**
13  * Clase principal que actua como el Orquestador o Controlador
    General de la aplicacion.
14  * Inicia el programa, configura los datos de prueba y maneja el
    bucle principal de ejecucion.
15  */
16  public class Main
17  {
18      public static void main(String[] args)
19      {
20          // =====
21          // 1. CONFIGURACION E INYECCION DE DEPENDENCIAS
22          // =====
23          // Creamos las dependencias clave una sola vez en el punto
            de entrada de la app
24          // y las compartimos (inyectamos) para evitar multiples
            instancias innecesarias.
25          ProductoService service = new ProductoService();
26          Scanner sc = new Scanner(System.in);
27
28          // Pasamos el Scanner y el Servicio al menu para que
            trabaje sobre las mismas instancias
29          MenuProducto menu = new MenuProducto(sc, service);
30
31          // Llenamos el inventario inicial con productos precargados
            para facilitar las pruebas
32          cargarDatosDePrueba(service);
33
34          // Variable local para almacenar y evaluar la decision del
            usuario en cada iteracion
35          int opcion;
36
37          // =====
38          // 2. BUCLE PRINCIPAL DE LA APLICACION (CONTROL DE FLUJO)
39          // =====
40          // El bloque do-while garantiza que el menu se muestre de
            entrada al menos una vez.
41          // El programa seguira corriendo de forma interactiva hasta
            que el usuario digite la opcion 6.
42          do
43          {
44              // Pinta las opciones disponibles en la terminal
45              menu.mostrarMenu();
46
47              // Lee la entrada numerica de forma segura delegando la
                validacion a la clase utilitaria
48              opcion = Validador.leerEntero(sc, "Elija una opcion: ")
                ;
49
50              // =====
51              // 3. CAPA DE SEGURIDAD Y MANEJO DE ERRORES (ROBUSTEZ)
52              // =====

```

```

53 // Todo el Switch-Expression se envuelve en un bloque
    try/catch global.
54 // Si cualquier operacion del menu o del servicio falla
    y lanza una excepcion,
55 // el flujo es capturado de inmediato. Esto evita que
    el programa "explote" (crash)
56 // y en su lugar muestra un mensaje elegante antes de
    reiniciar el bucle.
57 try
58 {
59     switch (opcion)
60     {
61         case 1 -> menu.agregarProducto();
62         case 2 -> menu.listarProductos();
63         case 3 -> menu.buscarProducto();
64         case 4 -> menu.actualizarProducto();
65         case 5 -> menu.eliminarProducto();
66         case 6 -> System.out.println("Hasta luego!");
67         default -> System.out.println("Opcion invalida.
            Elija un numero del 1 al 6.");
68     }
69 }
70 catch (ProductoNoEncontradoException |
    StockInsuficienteException e)
71 {
72     // Captura errores especificos del dominio del
        negocio (ej. ID inexistente o falta de stock)
73     System.out.println("\n[ERROR DE NEGOCIO]: " + e.
        getMessage());
74 }
75 catch (IllegalArgumentException e)
76 {
77     // Captura validaciones de datos genericos
        incorrectos (ej. campos vacios, valores
        negativos)
78     System.out.println("\n[ERROR DE VALIDACION]: " + e.
        getMessage());
79 }
80
81 // =====
82 // 4. CONTROL VISUAL Y REFRESCO DE PANTALLA
83 // =====
84 // Detiene temporalmente la ejecucion para que los
    mensajes de exito o error
85 // no desaparezcan de inmediato. Se ejecuta siempre y
    cuando no se elija salir.
86 if (opcion != 6)
87 {
88     presionarEnterParaContinuar(sc);
89 }
90
91 } while (opcion != 6); // Evalua la condicion de salida
    para ver si repite el ciclo

```

```

92
93     // Cierre preventivo del recurso Scanner al finalizar la
94     // ejecucion global del software
95     sc.close();
96 }
97
98 /**
99  * Metodo de Soporte: Alimenta la capa de servicios con
100  * productos quemados en codigo (mock data).
101  * @param service La instancia unica del servicio de productos
102  * donde se guardara la coleccion inicial.
103  */
104 private static void cargarDatosDePrueba(ProductoService service
105 )
106 {
107     service.guardar(new Producto("Cafe molido 500g", 4500, 30,
108     "Bebidas"));
109     service.guardar(new Producto("Yerba mate 1kg", 3200, 50, "
110     Bebidas"));
111     service.guardar(new Producto("Galletitas dulces", 1850,
112     100, "Almacen"));
113     service.guardar(new Producto("Aceite de oliva 500ml", 6700,
114     20, "Almacen"));
115     service.guardar(new Producto("Chocolate amargo 70%", 2900,
116     15, "Golosinas"));
117     System.out.println("Se cargaron 5 productos de prueba.\n");
118 }
119
120 /**
121  * Metodo de Soporte: Crea una pausa de consola obligando al
122  * usuario a presionar la tecla Enter.
123  * Esto evita que el menu secundario se encime velozmente sobre
124  * los resultados de las operaciones CRUD.
125  * @param sc La instancia compartida del Scanner de entrada.
126  */
127 private static void presionarEnterParaContinuar(Scanner sc)
128 {
129     System.out.println("\nPresione Enter para continuar...");
130     sc.nextLine(); // Bloquea la consola esperando unicamente
131     // el salto de linea del Enter
132 }
133 }

```

4.2. Validador.java

Listing 4: Clase Validador.java (Pre-entrega)

```

1 package util;
2
3 import java.util.InputMismatchException;
4 import java.util.Scanner;
5
6 import exception.StockInsuficienteException;

```

```

7
8 /**
9  * Clase Utilitaria (Utility Class): Provee metodos estaticos
    centralizados para validacion y lectura de datos.
10
11  *
12  * Sigue el patron Helper/Utility: Al declarar todos sus metodos
    como 'static', no requiere ser instanciada
13  * mediante el operador 'new'. Actua como una libreria interna que
    puede ser invocada directamente
14  * por cualquier capa de la aplicacion (UI o Service) bajo demanda.
15  */
16 public class Validador
17 {
18     // Nota de diseno: Idealmente, las clases utilitarias suelen
    llevar un constructor privado vacio
19     // para evitar explicitamente que alguien intente hacer:
    Validador v = new Validador();
20
21     // =====
22     // VALIDACIONES DE REGLAS DE NEGOCIO (Data Validation)
23     // =====
24     // Estos metodos aplican la tecnica de "Fail-Fast" (Fallar
    rapido). Si el parametro viola
25     // una restriccion, se interrumpe el hilo de inmediato lanzando
    una excepcion.
26     // No retornan nada (void): el silencio del metodo es sinonimo
    de que los datos son totalmente validos.
27
28     /**
29     * Valida que una cadena de texto contenga caracteres legibles
    y reales.
30     * @param nombre Cadena de texto a evaluar.
31     * @throws IllegalArgumentException Excepcion nativa si el
    texto es nulo o esta compuesto solo por espacios.
32     */
33     public static void validarNombre(String nombre)
34     {
35         // trim() remueve espacios fantasma en los extremos para
    evitar trampas como " "
36         if (nombre == null || nombre.trim().isEmpty())
37         {
38             throw new IllegalArgumentException("El nombre no puede
    estar vacio.");
39         }
40     }
41
42     /**
43     * Valida que los valores monetarios sean financieramente
    logicos.
44     * @param precio Valor decimal a evaluar.
45     * @throws IllegalArgumentException Si el precio es menor a
    cero (los productos gratis con valor 0 son validos).
46     */

```

```

46 public static void validarPrecio(double precio)
47 {
48     if (precio < 0)
49     {
50         throw new IllegalArgumentException("El precio no puede
51             ser negativo.");
52     }
53 }
54 /**
55  * Valida la existencia fisica del stock del inventario.
56  * @param stock Cantidad de unidades a evaluar.
57  * @throws StockInsuficienteException Excepcion personalizada
58  *     utilizada para reflejar un error semantico del negocio.
59  */
60 public static void validarStock(int stock) {
61     if (stock < 0)
62     {
63         throw new StockInsuficienteException("El stock no puede
64             ser negativo.");
65     }
66 }
67 /**
68  * Valida la categorizacion del catalogo.
69  * @param categoria Cadena de texto de la categoria.
70  * @throws IllegalArgumentException Si la categoria esta
71  *     ausente o vacia.
72  */
73 public static void validarCategoria(String categoria)
74 {
75     // isBlank() evalua de forma optima si la cadena tiene
76     // longitud cero o puros espacios en blanco
77     if (categoria == null || categoria.isBlank())
78     {
79         throw new IllegalArgumentException("La categoria no
80             puede estar vacia.");
81     }
82 }
83 // =====
84 // METODOS DE CAPTURA ROBUSTA POR CONSOLA (Console Input
85 // Processing)
86 // =====
87 // Resuelven el problema clasico del Scanner en Java: el
88 // desbordamiento o bloqueo del buffer
89 // cuando el usuario introduce caracteres no numericos o
90 // presiona saltos de linea imprevistos.
91 /**
92  * Captura de forma blindada un valor entero, atrapando errores
93  * tipograficos del usuario.
94  *

```

```

89      * @param sc Instancia unica y compartida del Scanner.
90      * @param mensaje Indicacion textual (Prompt) que guia al
          usuario sobre que ingresar.
91      * @return El numero entero ya validado sintacticamente.
92      */
93      public static int leerEntero(Scanner sc, String mensaje)
94      {
95          // Bucle infinito controlado: No se rompera hasta que el
          bloque 'try' alcance con exito el 'return'
96          while (true)
97          {
98              System.out.println(mensaje);
99              try
100              {
101                  int valor = sc.nextInt();
102
103                  // LIMPIEZA CLAVE DEL BUFFER: nextInt() consume el
                  numero pero deja el caracter invisible
104                  // del salto de linea ('\n') en la consola. sc.
                  nextLine() consume ese residuo para evitar
105                  // que el siguiente metodo de lectura se salte
                  automaticamente.
106                  sc.nextLine();
107
108                  return valor; // Rompe el bucle e inmuniza la
                  salida
109              }
110              catch (InputMismatchException e)
111              {
112                  System.out.println("Debe ingresar un numero entero.
                  Intente nuevamente.");
113
114                  // RECOLECCION DE BASURA DEL BUFFER: Si el usuario
                  escribe una letra (ej. "hola"),
115                  // nextInt() falla y deja la palabra "hola"
                  atrapada en el buffer. Si no llamamos a nextLine
                  ()
116                  // aqui, el bucle intentara leer "hola"
                  infinitamente provocando un ciclo sin fin.
117                  sc.nextLine();
118              }
119          }
120      }
121
122      /**
123       * Captura de forma blindada un valor decimal, previniendo
          fallas de localizacion por comas o puntos.
124       *
125       * @param sc Instancia compartida del Scanner.
126       * @param mensaje Indicacion en pantalla.
127       * @return El numero de punto flotante de doble precision (
          double) validado.
128       */

```

```

129 public static double leerDouble(Scanner sc, String mensaje)
130 {
131     while (true)
132     {
133         System.out.println(mensaje);
134         try
135         {
136             double valor = sc.nextDouble();
137             sc.nextLine(); // Consume el residuo del salto de
                           linea ('\n')
138             return valor;
139         }
140         catch (Exception e)
141         {
142             // Maneja genericamente cualquier anomalia
                           tipografica (ej. ingresar un punto cuando el
                           sistema espera coma)
143             System.out.println("Debe ingresar un numero decimal
                           usando el separador correcto.");
144             sc.nextLine(); // Limpia la entrada corrupta del
                           buffer antes de reiniciar el ciclo
145         }
146     }
147 }
148
149 /**
150  * Lee de forma directa lineas completas de caracteres sin
                           riesgo de saltos automaticos no deseados.
151  *
152  * @param sc Instancia compartida del Scanner.
153  * @param mensaje Indicacion en pantalla.
154  * @return La cadena de caracteres digitada por el usuario.
155  */
156 public static String leerTexto(Scanner sc, String mensaje)
157 {
158     System.out.println(mensaje);
159     // Al usar nextLine() directamente tras haber mantenido
                           limpios los buffers numericos anteriores,
160     // garantizamos una captura limpia sin efectos secundarios.
161     return sc.nextLine();
162 }
163 }

```

4.3. Producto.java

Listing 5: Clase Producto.java (Pre-entrega)

```

1 package model;
2
3 /**
4  * Modelo de Dominio o Plain Old Java Object (POJO): Representa la
                           entidad basica de un Producto.
5  *

```

```

6  * Aplica el principio de Encapsulamiento: Los atributos son
   * privados y el acceso se restringe
7  * a traves de metodos publicos (getters y setters). Esta clase
   * sigue el Principio de Responsabilidad
8  * Unica (SRP), limitandose a modelar un objeto del mundo real; no
   * gestiona bases de datos ni logica visual.
9  */
10 public class Producto
11 {
12     // =====
13     // ATRIBUTOS (Estado del Objeto)
14     // =====
15     // Al ser 'private', se protege la integridad de los datos
   * impidiendo modificaciones
16     // externas maliciosas o directas que puedan romper el estado
   * de la entidad.
17     private int id;
18     private String nombre;
19     private double precio;
20     private int stock;
21     private String categoria;
22
23     // =====
24     // CONSTRUCTORES (Instanciacion)
25     // =====
26
27     /**
28      * Constructor Parametrizado (Sin ID).
29      * Se utiliza para crear productos nuevos antes de ser
   * persistidos. El ID no se incluye
30      * aqui porque su generacion es responsabilidad de la capa de
   * persistencia (Base de datos / Servicio).
31      */
32     public Producto(String nombre, double precio, int stock, String
   categoria)
33     {
34         this.nombre = nombre;
35         this.precio = precio;
36         this.stock = stock;
37         this.categoria = categoria;
38     }
39
40     /**
41      * Constructor Vacio (Por defecto).
42      * Requisito fundamental de la especificacion JavaBeans.
   * Permite instanciar objetos "vacios"
43      * para completarlos progresivamente mediante Setters. Es
   * obligatorio para que frameworks
44      * de mapeo como Hibernate/JPA, Jackson (JSON) o Spring puedan
   * realizar la instanciacion por reflexion.
45      */
46     public Producto()
47     {

```

```

48         // Bloque intencionalmente vacio para cumplir con el
           estandar POJO/JavaBean
49     }
50
51     // =====
52     // METODOS DE ACCESO (Getters y Setters)
53     // =====
54     // Constituyen la interfaz publica segura para interactuar con
           los atributos privados.
55
56     public int getId()
57     {
58         return id;
59     }
60
61     public void setId(int id)
62     {
63         this.id = id;
64     }
65
66     public String getNombre()
67     {
68         return nombre;
69     }
70
71     public void setNombre(String nombre)
72     {
73         this.nombre = nombre;
74     }
75
76     public double getPrecio()
77     {
78         return precio;
79     }
80
81     public void setPrecio(double precio)
82     {
83         this.precio = precio;
84     }
85
86     public int getStock()
87     {
88         return stock;
89     }
90
91     public void setStock(int stock)
92     {
93         this.stock = stock;
94     }
95
96     public String getCategoria()
97     {
98         return categoria;

```

```

99     }
100
101     public void setCategoria(String categoria)
102     {
103         this.categoria = categoria;
104     }
105
106     // =====
107     // METODOS DE REPRESENTACION Y SALIDA
108     // =====
109
110     /**
111      * Sobreescritura del metodo estandar toString() heredado de la
112      * clase base java.lang.Object.
113      * Convierte la direccion de memoria nativa del objeto en una
114      * cadena de texto util y legible.
115      * Ideal para tareas de depuracion (debugging) o
116      * concatenaciones rapidas de texto.
117      * @return Una representacion formateada con el estado actual
118      * de los atributos del producto.
119      */
120     @Override
121     public String toString()
122     {
123         return "ID: " + id +
124             " | " + nombre +
125             " | $" + precio +
126             " | Stock: " + stock +
127             " | Categoria: " + categoria;
128     }
129
130     /**
131      * Envia directamente la informacion formateada del producto
132      * hacia el flujo de salida estandar (Consola).
133      * Diseñado específicamente para estandarizar la visualizacion
134      * de registros individuales dentro de la UI.
135      */
136     public void mostrar()
137     {
138         System.out.println("ID: " + id +
139             " | " + nombre +
140             " | $" + precio +
141             " | Stock: " + stock +
142             " | Categoria: " + categoria);
143     }
144 }

```

4.4. ProductoService.java

Listing 6: Clase ProductoService.java (Pre-entrega)

```

1 package service;
2

```

```

3 import java.util.ArrayList;
4 import java.util.List;
5
6 import exception.ProductoNoEncontradoException;
7 import model.Producto;
8 import util.Validador;
9
10 /**
11  * Capa de Servicio (Service Layer): Orquesta e implementa la
12   * Logica de Negocio (Business Logic) del sistema.
13  *
14  * Es la capa intermedia que actua como barrera de seguridad entre
15   * la Interfaz de Usuario (UI)
16   * y la persistencia de datos (en este caso, una coleccion en
17   * memoria).
18  *
19  * Responsabilidades:
20  * - Gestionar el ciclo de vida del almacen de datos (List).
21  * - Controlar la generacion e integridad de llaves primarias
22   * unicas (IDs).
23  * - Forzar el cumplimiento de las reglas o restricciones de
24   * negocio mediante validaciones antes de cualquier mutacion.
25  *
26  * Cumple con el principio de Desacoplamiento (Alta Cohesion / Bajo
27   * Acoplamiento): Al no contener codigo
28   * acoplado a la consola (sin Scanner ni System.out), esta misma
29   * clase podra ser reutilizada intacta en el futuro
30   * cuando el sistema migre a una arquitectura Web o API REST con
31   * Spring Boot.
32  */
33 public class ProductoService
34 {
35     // Almacenamiento volatil en memoria utilizando una lista
36     // dinamica (Estructura de Datos).
37     // Se encapsula como 'private' para que ninguna clase externa
38     // altere la coleccion sin pasar por las reglas del servicio.
39     private List<Producto> productos = new ArrayList<>();
40
41     // Generador secuencial de IDs unicos. Al ser un atributo
42     // estatico (static), su valor se conserva a nivel
43     // de Clase y no de Instancia, garantizando consistencia
44     // matematica y evitando la duplicidad de identificadores.
45     private static int contadorId = 1;
46
47     // =====
48     // OPERACIONES CRUD (Create, Read, Update, Delete)
49     // =====
50
51     /**
52      * CREATE: Procesa, valida y persiste una nueva entidad
53      * Producto en el sistema.
54      * Si alguna validacion de datos falla, se dispara una
55      * RuntimeException interrumpiendo el flujo,

```

```

42     * lo que garantiza que nunca ingresen datos corruptos a la
43     lista.
44     *
45     * @param p Objeto producto transitorio (aun sin ID).
46     * @return El objeto Producto completamente persistido (con su
47     ID oficial asignado).
48     */
49     public Producto guardar(Producto p)
50     {
51         // Fase de Validacion: Filtro estricto que evalua las
52         restricciones del negocio antes de impactar los datos
53         Validador.validarNombre(p.getNombre());
54         Validador.validarPrecio(p.getPrecio());
55         Validador.validarStock(p.getStock());
56         Validador.validarCategoria(p.getCategoria());
57
58         // Asignacion de Identificador: Mecanismo de persistencia
59         controlado internamente por el backend.
60         // Se aplica un post-incremento (contadorId++) para
61         preparar automaticamente el valor del siguiente registro
62         .
63         p.setId(contadorId);
64         contadorId++;
65
66         // Insercion en la estructura de datos en memoria
67         productos.add(p);
68
69         return p;
70     }
71
72     /**
73     * READ (Coleccion): Expone la lista completa de elementos
74     registrados.
75     * @return List de objetos Producto actualmente almacenados en
76     memoria.
77     */
78     public List<Producto> listarTodos()
79     {
80         return productos;
81     }
82
83     /**
84     * READ (Individual): Implementa un algoritmo de busqueda
85     lineal para localizar un registro por su ID unico.
86     *
87     * @param id Identificador numerico del producto deseado.
88     * @return El objeto Producto que coincide exactamente con el
89     parametro recibido.
90     * @throws ProductoNoEncontradoException Excepcion
91     personalizada si el bucle termina sin hallar coincidencias.
92     */
93     public Producto obtenerPorId(int id)
94     {

```

```

84     for (Producto p : productos)
85     {
86         if (p.getId() == id)
87         {
88             return p; // Retorno inmediato (Early Return) al
                        encontrar el objetivo, optimizando el ciclo
89         }
90     }
91
92     // Control de errores de negocio: Si el flujo llega a este
        punto, significa que el ID es invalido
93     throw new ProductoNoEncontradoException("No se encontro un
        producto con el ID " + id);
94 }
95
96 /**
97  * UPDATE: Sobreescribe el estado de un registro existente
        basandose en un ID de destino.
98  *
99  * Nota de Arquitectura: El parametro datos actua tecnicamente
        como un DTO (Data Transfer Object),
100  * un contenedor temporal que transporta los nuevos datos desde
        la UI hacia la logica de actualizacion.
101  *
102  * @param id Identificador del producto real que se desea
        modificar.
103  * @param datos Objeto con la informacion actualizada que se va
        a inyectar.
104  * @return La entidad Producto con su nuevo estado consolidado.
105  */
106 public Producto actualizar(int id, Producto datos)
107 {
108     // Reutilizacion de codigo: Se delega la busqueda a
        obtenerPorId.
109     // Si el ID no existe, este metodo lanza su excepcion
        nativa abortando el proceso de forma segura.
110     Producto p = obtenerPorId(id);
111
112     // Validamos exhaustivamente el paquete de cambios antes de
        mutar el objeto original
113     Validador.validarNombre(datos.getNombre());
114     Validador.validarPrecio(datos.getPrecio());
115     Validador.validarStock(datos.getStock());
116     Validador.validarCategoria(datos.getCategoria());
117
118     // Mutacion de estado por Referencia: Debido a que Java
        manipula las referencias a objetos en memoria,
119     // al alterar las propiedades de 'p', los cambios impactan
        directamente sobre el elemento dentro de la List.
120     // No es necesario remover e insertar el elemento de nuevo.
121     p.setNombre(datos.getNombre());
122     p.setPrecio(datos.getPrecio());
123     p.setStock(datos.getStock());

```

```

124         p.setCategoria(datos.getCategoria());
125
126         return p;
127     }
128
129     /**
130      * DELETE: Remueve fisicamente un registro del almacen de datos
131      * basandose en su ID.
132      * @param id Identificador numerico del objeto a dar de baja.
133      */
134     public void eliminar(int id)
135     {
136         // Localiza el objeto en memoria. Si no existe, detiene la
137         // operacion lanzando la excepcion correspondiente.
138         Producto p = obtenerPorId(id);
139
140         // Remocion fisica: Rompe el enlace del objeto dentro del
141         // ArrayList
142         productos.remove(p);
143     }
144
145     /**
146      * Metodo exclusivo para entornos de prueba.
147      * Restablece el contador estatico de identificadores y vacia
148      * la lista
149      * para asegurar la independencia de los tests unitarios.
150      */
151     public void reiniciarServicio()
152     {
153         productos.clear();
154         contadorId = 1;
155     }
156 }

```

4.5. ProductoNoEncontradoException.java

Listing 7: Clase ProductoNoEncontradoException.java (Pre-entrega)

```

1 package exception;
2
3 /**
4  * Excepcion Personalizada de Negocio (Custom Domain Exception).
5  * Se dispara exclusivamente cuando un proceso del sistema intenta
6  * interactuar con un ID de producto inexistente.
7  *
8  * Arquitectura de Errores:
9  * Hereda de java.lang.RuntimeException, clasificandose como una
10  * Unchecked Exception (Excepcion No Verificada).
11  * A diferencia de las Checked Exceptions (que heredan directamente
12  * de java.lang.Exception), las excepciones no
13  * verificadas no obligan a propagar firmas con la palabra clave '
14  * throws' ni fuerzan el uso de bloques try/catch

```

```

11  * en todas las capas intermedias, manteniendo el codigo limpio y
    * desacoplado.
12  *
13  * Ventajas del Diseno Expresivo (Domain-Driven Design):
14  * Provee una semantica clara y especifica en la pila de llamadas (
    * Stacktrace). Es mucho mas descriptivo
15  * leer un error tipo 'ProductoNoEncontradoException' que uno
    * generico como 'IllegalArgumentException' o '
    * NullPointerException'.
16  */
17  public class ProductoNoEncontradoException extends RuntimeException
18  {
19      /**
20       * Constructor de la excepcion personalizada.
21       *
22       * @param mensaje Texto detallado que describe las
23       * circunstancias exactas de la anomalia (ej. el ID fallido).
24       */
25      public ProductoNoEncontradoException(String mensaje)
26      {
27          // La palabra clave 'super' invoca de forma explicita al
28          // constructor parametrizado de la clase
29          // base (RuntimeException). Esto inicializa los mecanismos
30          // nativos de Java para registrar el mensaje
31          // de error y congelar el estado actual del hilo,
32          // permitiendo luego recuperarlo mediante e.getMessage().
33          super(mensaje);
34      }
35  }

```

4.6. StockInsuficienteException.java

Listing 8: Clase StockInsuficienteException.java (Pre-entrega)

```

1  package exception;
2
3  /**
4   * Excepcion Personalizada de Negocio (Custom Domain Exception).
5   * Se dispara cuando una operacion viola las reglas de consistencia
6   * de inventario, ya sea al ingresar
7   * valores de stock fisicamente imposibles (negativos) o al
8   * intentar egresar mas unidades de las disponibles.
9   *
10  * Arquitectura de Errores:
11  * Al igual que 'ProductoNoEncontradoException', hereda de java.
12  * lang.RuntimeException, consolidandose
13  * como una Unchecked Exception (Excepcion No Verificada). Esto
14  * permite un flujo libre de firmas en la capa
15  * de servicios ('ProductoService') y centraliza de forma elegante
16  * su captura en el controlador principal ('Main.java').
17  *
18  * Polimorfismo Semantico:

```

```

14  * Su diseño es lo suficientemente versátil para actuar en dos
    * frentes del dominio del negocio:
15  * 1. Fase de Administración (Backoffice): Previene errores de
    * carga (ej. ingresar un stock inicial de -10).
16  * 2. Fase de Transacción (E-Commerce): Servirá como la base
    * perfecta para el control del flujo cuando
17  * el sistema evolucione y requiera validar un Carrito de
    * Compras (ej. un cliente pide 5 unidades pero quedan 2).
18  */
19  public class StockInsuficienteException extends RuntimeException
20  {
21      /**
22       * Constructor de la excepción personalizada de stock.
23       *
24       * @param mensaje Texto descriptivo que detalla la
25       * inconsistencia de inventario detectada por el sistema.
26       */
27      public StockInsuficienteException(String mensaje)
28      {
29          // La palabra clave 'super' delega la inicialización del
30          // mensaje al constructor de RuntimeException.
31          // Esto indexa la cadena de texto en las propiedades
32          // nativas del objeto arrojable (Throwable),
33          // quedando disponible para ser capturado en las capas
34          // superiores de la interfaz de usuario.
35          super(mensaje);
36      }
37  }

```

4.7. MenuProducto.java

Listing 9: Clase MenuProducto.java (Pre-entrega)

```

1  package ui;
2
3  import java.util.List;
4  import java.util.Scanner;
5
6  import model.Producto;
7  import service.ProductoService;
8  import util.Validador;
9
10 /*
11  * Manejar la interacción con el usuario: mostrar el menú, leer
12  * datos, mostrar mensajes de error o éxito.
13
14  * Responsable de:
15  *   - Mostrar el menú al usuario
16  *   - Pedirle los datos
17  *   - Mostrar los resultados
18
19  * No contiene la lógica del negocio.
20  * No controla el flujo del programa.

```

```

20 */
21
22 public class MenuProducto
23 {
24     // Atributos de la clase: el Scanner y el Service se reciben
25     // por constructor (no se crean aqui dentro).
26     // Esto permite que quien instancie la clase tenga el control
27     // sobre que instancias usar.
28     private final Scanner sc;
29     private final ProductoService service;
30
31     /**
32      * Constructor de la clase.
33      * Implementa el patron de "Inyeccion de Dependencias por
34      * Constructor" (muy utilizado en Spring Boot).
35      * @param sc Objeto Scanner compartido para la lectura de datos
36      * por consola.
37      * @param service Capa de servicio encargada de la logica del
38      * negocio de los productos.
39      */
40     public MenuProducto(Scanner sc, ProductoService service)
41     {
42         this.sc = sc;
43         this.service = service;
44     }
45
46     /**
47      * Muestra visualmente las opciones disponibles en la consola.
48      * Incluye saltos de linea al inicio para limpiar y refrescar
49      * el aspecto de la pantalla.
50      */
51     public void mostrarMenu()
52     {
53         System.out.println("\n\n\n"); // Empuja el texto anterior
54         // hacia arriba para simular una pantalla limpia
55         System.out.println("===== TechLab - Gestion de Productos
56         =====");
57         System.out.println("1) Agregar producto");
58         System.out.println("2) Listar productos");
59         System.out.println("3) Buscar producto por ID");
60         System.out.println("4) Actualizar producto");
61         System.out.println("5) Eliminar producto");
62         System.out.println("6) Salir");
63         System.out.println("
64         =====");
65     }
66
67     // =====
68     // Operaciones del CRUD (Opciones del menu)
69     // =====
70
71     /**

```

```

63      * Opcion 1: Solicita los datos de un nuevo producto, lo
        construye y delega el guardado al servicio.
64      */
65      public void agregarProducto()
66      {
67          System.out.println("- Nuevo producto -");
68
69          // Se utiliza la clase utilitaria 'Validador' para
            garantizar que los tipos de datos ingresados sean
            correctos
70          String nombre = Validador.leerTexto(sc, "Nombre: ");
71          double precio = Validador.leerDouble(sc, "Precio: ");
72          int stock = Validador.leerEntero(sc, "Stock: ");
73          String categoria = Validador.leerTexto(sc, "Categoria: ");
74
75          // Construimos el objeto producto de forma temporal con los
            datos capturados
76          Producto p = new Producto(nombre, precio, stock, categoria)
            ;
77
78          // Enviamos el producto al servicio. El metodo 'guardar' se
            encarga internamente de validar las
79          // reglas de negocio (ej. precios no negativos) y de
            asignarle un ID unico de forma automatica.
80          Producto guardado = service.guardar(p);
81
82          // Confirmamos el exito de la operacion mostrando el ID
            generado por el sistema
83          System.out.println("Producto agregado con el id " +
            guardado.getId());
84      }
85
86      /**
87      * Opcion 2: Recupera la lista completa de productos desde el
            servicio y los muestra en pantalla.
88      */
89      public void listarProductos()
90      {
91          // Solicitamos la lista de productos almacenada en la capa
            de negocio
92          List<Producto> lista = service.listarTodos();
93
94          // Validacion local de interfaz: Si la lista viene vacia,
            informamos al usuario y cortamos la ejecucion del metodo
95          if (lista.isEmpty()){
96              System.out.println("No hay productos cargados.");
97              return;
98          }
99
100         System.out.println("- Catalogo -");
101         // Recorremos la lista usando un bucle for-each para
            imprimir cada producto de manera estructurada
102         for(Producto p : lista){

```

```

103         p.mostrar(); // Llama al metodo interno del modelo
           diseñado para imprimir con formato propio
104     }
105 }
106
107 /**
108  * Opcion 3: Solicita un ID numerico e intenta recuperar el
           producto correspondiente mediante el servicio.
109  */
110 public void buscarProducto()
111 {
112     // Solicitamos de forma segura un numero entero para el ID
113     int id = Validador.leerEntero(sc, "Ingrese el id del
           producto: ");
114
115     // El servicio buscara el producto. Si no existe, lanzara
           una excepcion (ProductoNoEncontradoException)
116     // que sera capturada y manejada por el try/catch ubicado
           en la clase Main.java
117     Producto p = service.obtenerPorId(id);
118
119     // Si el producto fue encontrado con exito, se muestra su
           representacion en texto
120     System.out.println("Encontrado: " + p);
121 }
122
123 /**
124  * Opcion 4: Busca un producto existente y actualiza todas sus
           propiedades con nuevos valores provistos.
125  */
126 public void actualizarProducto()
127 {
128     int id = Validador.leerEntero(sc, "Ingrese el id del
           producto a actualizar: ");
129
130     // Recuperamos y mostramos primero los datos actuales para
           que el usuario visualice que esta modificando.
131     // Al igual que en buscarProducto, si el ID no existe, el
           flujo se interrumpe aqui por la excepcion del servicio.
132     Producto actual = service.obtenerPorId(id);
133     System.out.println("Datos actuales: " + actual);
134
135     System.out.println("- Ingrese los nuevos datos -");
136     String nombre = Validador.leerTexto(sc, "Nombre: ");
137     double precio = Validador.leerDouble(sc, "Precio: ");
138     int stock = Validador.leerEntero(sc, "Stock: ");
139     String categoria = Validador.leerTexto(sc, "Categoria: ");
140
141     // Creamos un nuevo objeto con los cambios requeridos
142     Producto datos = new Producto(nombre, precio, stock,
           categoria);
143

```

```

144         // Enviamos el ID del destino y el objeto con las
145         modificaciones al servicio para que procese el cambio
146         Producto actualizado = service.actualizar(id, datos);
147
148         // Confirmamos mostrando en pantalla el estado final del
149         producto modificado
150         System.out.println("Producto actualizado: " + actualizado);
151     }
152
153     /**
154     * Opcion 5: Solicita un ID numerico y delega la remocion
155     fisica o logica del producto al servicio.
156     */
157     public void eliminarProducto()
158     {
159         int id = Validador.leerEntero(sc, "Ingrese el id del
160         producto a eliminar: ");
161
162         // El servicio valida la existencia del ID y procede a
163         borrarlo del almacen de datos
164         service.eliminar(id);
165
166         // Si el servicio no arrojo ninguna excepcion, confirmamos
167         la eliminacion exitosa
168         System.out.println("Producto eliminado.");
169     }
170 }

```

5. Corridas



```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
1
--- Nuevo producto ---
Nombre:
Monster 500cc
Precio:
3500
Stock:
25
Categoria:
Bebidas
Producto agregado con el id 6
Presione Enter para continuar...

```

Figura 3: Elección de la opción 1

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
2
--- Catalogo ---
ID: 1 | Cafe molido 500g | $4500.0 | Stock: 30 | Categoria: Bebidas
ID: 2 | Yerba mate 1kg | $3200.0 | Stock: 50 | Categoria: Bebidas
ID: 3 | Galletitas dulces | $1850.0 | Stock: 100 | Categoria: Almacen
ID: 4 | Aceite de oliva 500ml | $6700.0 | Stock: 20 | Categoria: Almacen
ID: 5 | Chocolate amargo 70% | $2900.0 | Stock: 15 | Categoria: Golosinas
ID: 6 | Monster 500cc | $3500.0 | Stock: 25 | Categoria: Bebidas
Presione Enter para continuar...

```

Figura 4: Elección de la opción 2

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
3
Ingrese el id del producto:
3
Encontrado: ID: 3 | Galletitas dulces | $1850.0 | Stock: 100 | Categoria: Almacen
Presione Enter para continuar...

```

Figura 5: Elección de la opción 3

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
4
Ingrese el id del producto a actualizar:
6
Datos actuales: ID: 6 | Monster 500cc | $3500.0 | Stock: 25 | Categoria: Bebidas
--- Ingrese los nuevos datos ---
Nombre:
Monster Mango Loco 500cc
Precio:
3000
Stock:
15
Categoria:
Bebida
Producto actualizado: ID: 6 | Monster Mango Loco 500cc | $3000.0 | Stock: 15 | Categoria: Bebida
Presione Enter para continuar...

```

Figura 6: Elección de la opción 4

```
===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
5
Ingrese el id del producto a eliminar:
1
Producto eliminado.
Presione Enter para continuar...
```

Figura 7: Elección de la opción 5

```
===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
6
Hasta luego!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 03:24 min
[INFO] Finished at: 2026-10-06T21:24:47-03:00
[INFO] -----
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGH:~/JAVA/java-preentrega$
```

Figura 8: Elección de la opción 6

6. Pruebas Unitarias

Las pruebas unitarias implementadas en este proyecto se enfocaron exclusivamente en el modulo ProductoService debido a que este componente constituye el núcleo de la lógica de negocio del sistema. En una arquitectura de software limpia y distribuida por capas, es fundamental garantizar la estabilidad y el correcto comportamiento de la capa de servicio de manera aislada.

Este enfoque selectivo se justifica por las siguientes razones de ingeniería de software:

- **Aislamiento de la Logica de Negocio:** Las clases pertenecientes al paquete model solo actuan como contenedores de datos (POJOs), mientras que las utilidades del paquete util y las excepciones son componentes de soporte. ProductoService es el unico modulo que procesa, altera y valida las reglas del negocio.
- **Desacoplamiento de la Interfaz:** El paquete ui (MenuProducto) y la clase Main se encargan unicamente de la interaccion con el usuario a traves de la consola. Probar esos modulos requeriría simular entradas de teclado (Scanner) y flujos de salida (System.out), lo cual pertenece al ambito de las pruebas de integracion o de interfaz, no a las pruebas unitarias.
- **Validacion de Flujos Criticos:** Al concentrar los esfuerzos en este servicio, se logro verificar de forma automatizada tanto los casos de exito del CRUD como la correcta respuesta del sistema ante fallos, forzando de manera segura el lanzamiento de las excepciones ProductoNoEncontradoException y StockInsuficienteException.

6.1. Estrategia de Testing y Robustez del Dominio

Para asegurar la estabilidad matemática del software, se incorporaron técnicas avanzadas de control de estado en la suite de pruebas unitarias:

- **Mecánica Fail-Fast:** Los métodos de prueba evalúan que los validadores lancen excepciones de manera inmediata (`IllegalArgumentException`) ante cargas de datos corruptas o inconsistentes (ej. nombres vacíos o precios negativos), protegiendo la integridad del catálogo en memoria.
- **Control de Estado Estático:** Debido a que la capa de servicio utiliza un contador estático secuencial para la asignación de llaves primarias (IDs), se diseñó un método utilitario `reiniciarServicio()` invocado en el ciclo `@BeforeEach`. Esto limpia la colección y resetea el puntero a 1 antes de cada test, garantizando el principio de Independencia Funcional (las pruebas no se contaminan entre sí).

6.2. ProductoServiceTest.java

Listing 10: Clase `ProductoServiceTest.java` (Pre-entrega)

```
1 package com.josepaz.preentrega;
2
3 // IMPORTS ESTATICOS: Permiten usar los metodos de verificacion de
4   JUnit 5
5 import static org.junit.jupiter.api.Assertions.assertEquals;
6 import static org.junit.jupiter.api.Assertions.assertNotNull;
7 import static org.junit.jupiter.api.Assertions.assertThrows;
8 import static org.junit.jupiter.api.Assertions.assertTrue;
9
10 import java.util.List;
11
12 import org.junit.jupiter.api.AfterEach;
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15 import org.junit.jupiter.api.TestInfo;
16
17 // NUEVOS IMPORTS: Enlazan las clases de negocio con la suite de
18   pruebas
19 import service.ProductoService;
20 import model.Producto;
21 import exception.ProductoNoEncontradoException;
22 import exception.StockInsuficienteException;
23
24 /**
25  * Clase de pruebas unitarias para ProductoService utilizando JUnit
26  * 5.
27  * Aplica el patron internacional AAA (Arrange, Act, Assert).
28  */
29 public class ProductoServiceTest
30 {
31     private ProductoService service;
32
33     /**
34      * Ciclo de Vida - Configuracion Inicial (Setup).
```

```

32     * Se ejecuta antes de cada metodo de prueba individual para
        limpiar el estado de la memoria.
33     * Invoca reiniciarServicio para restablecer tambien el
        contador secuencial de IDs estaticos.
34     */
35     @BeforeEach
36     public void setUp()
37     {
38         service = new ProductoService();
39         service.reiniciarServicio(); // NUEVO: Garantiza que el
            primer ID asignado sea siempre 1
40     }
41
42     /**
43     * Ciclo de Vida - Despliegue de Resultados (Teardown).
44     * Se ejecuta inmediatamente despues de que un test finaliza
        con exito.
45     * Utiliza TestInfo para obtener el nombre del metodo y
        mostrarlo en la terminal de Linux.
46     */
47     @AfterEach
48     public void alFinalizarCadaTest(TestInfo testInfo)
49     {
50         System.out.println(" -> OK: Test aprobado con exito: " +
            testInfo.getTestMethod().get().getName() + "());
51     }
52
53     // =====
54     // TESTS PARA LA OPERACION: GUARDAR (CREATE)
55     // =====
56
57     @Test
58     public void guardarProducto_Exitoso()
59     {
60         Producto nuevo = new Producto("Teclado Mecanico", 8500.0,
            15, "Electronica");
61         Producto guardado = service.guardar(nuevo);
62
63         assertNotNull(guardado);
64         assertEquals(1, guardado.getId());
65         assertEquals("Teclado Mecanico", guardado.getNombre());
66         assertEquals(15, guardado.getStock());
67     }
68
69     @Test
70     public void guardarProducto_Error_NombreVacio_LanzaException()
71     {
72         Producto invalido = new Producto(" ", 500.0, 10, "Almacen
            ");
73
74         assertThrows(IllegalArgumentException.class, () -> {
75             service.guardar(invalido);
76         });

```

```

77     }
78
79     @Test
80     public void guardarProducto_Error_PrecioNegativo_LanzaException
81         ()
82     {
83         Producto invalido = new Producto("Mouse", -150.0, 5, "
84             Electronica");
85
86         assertThrows(IllegalArgumentException.class, () -> {
87             service.guardar(invalido);
88         });
89     }
90
91     @Test
92     public void
93         guardarProducto_Error_StockNegativo_LanzaStockInsuficienteException
94         ()
95     {
96         Producto invalido = new Producto("Monitor", 45000.0, -1, "
97             Electronica");
98
99         assertThrows(StockInsuficienteException.class, () -> {
100             service.guardar(invalido);
101         });
102     }
103
104     // =====
105     // TESTS PARA LA OPERACION: OBTENER POR ID / LISTAR (READ)
106     // =====
107
108     @Test
109     public void obtenerPorId_Exitoso()
110     {
111         Producto p = service.guardar(new Producto("Gorra", 1200.0,
112             20, "Indumentaria"));
113         Producto encontrado = service.obtenerPorId(p.getId());
114
115         assertNotNull(encontrado);
116         assertEquals(p.getId(), encontrado.getId());
117     }
118
119     @Test
120     public void
121         obtenerPorId_Error_IdInexistente_LanzaProductoNoEncontradoException
122         ()
123     {
124         assertThrows(ProductoNoEncontradoException.class, () -> {
125             service.obtenerPorId(999);
126         });
127     }
128
129     @Test

```

```

122 public void listarTodos_Exitoso()
123 {
124     service.guardar(new Producto("A", 10.0, 5, "Cat"));
125     service.guardar(new Producto("B", 20.0, 10, "Cat"));
126
127     List<Producto> lista = service.listarTodos();
128     assertEquals(2, lista.size());
129 }
130
131 // =====
132 // TESTS PARA LA OPERACION: ACTUALIZAR (UPDATE)
133 // =====
134
135 @Test
136 public void actualizarProducto_Exitoso()
137 {
138     Producto original = service.guardar(new Producto("Original"
139         , 100.0, 5, "Cat"));
140     Producto nuevosDatos = new Producto("Modificado", 150.0, 8,
141         "NuevaCat");
142
143     Producto actualizado = service.actualizar(original.getId(),
144         nuevosDatos);
145
146     assertEquals("Modificado", actualizado.getNombre());
147     assertEquals(150.0, actualizado.getPrecio());
148     assertEquals(8, actualizado.getStock());
149 }
150
151 // =====
152 // TESTS PARA LA OPERACION: ELIMINAR (DELETE)
153 // =====
154
155 @Test
156 public void eliminarProducto_Exitoso()
157 {
158     Producto p = service.guardar(new Producto("Eliminar", 50.0,
159         2, "Cat"));
160     int id = p.getId();
161
162     service.eliminar(id);
163
164     assertTrue(service.listarTodos().isEmpty());
165     assertThrows(ProductoNoEncontradoException.class, () -> {
166         service.obtenerPorId(id);
167     });
168 }
169 }

```

```
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM: ~/JAVA/java-preentrega$ ./test.sh preentrega
=====
Ejecutando Compilacion del Proyecto - TechLab
=====
[INFO] Scanning for projects...
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ hola-mundo-terminal ---
[INFO] Deleting /home/josepaz/JAVA/java-preentrega/target
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 7 source files with javac [debug target 21] to target/classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.945 s
[INFO] Finished at: 2026-10-06T22:26:07-03:00
[INFO] -----
=====
Ejecutando las Pruebas Unitarias con JUnit 5
=====
[INFO] Scanning for projects...
```

Figura 9: Como ejecutar las pruebas unitarias

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.josepaz.preentrega.ProductoServiceTest
-> OK: Test aprobado con éxito: obtenerPorId_Error_IdInexistente_LanzaProductoNoEncontradoException()
-> OK: Test aprobado con éxito: guardarProducto_Error_PrecioNegativo_LanzaException()
-> OK: Test aprobado con éxito: actualizarProducto_Exitoso()
-> OK: Test aprobado con éxito: guardarProducto_Error_NombreVacio_LanzaException()
-> OK: Test aprobado con éxito: guardarProducto_Exitoso()
-> OK: Test aprobado con éxito: eliminarProducto_Exitoso()
-> OK: Test aprobado con éxito: listarTodos_Exitoso()
-> OK: Test aprobado con éxito: obtenerPorId_Exitoso()
-> OK: Test aprobado con éxito: guardarProducto_Error_StockNegativo_LanzaStockInsuficienteException()
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.347 s -- in com.josepaz.preentrega
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.789 s
[INFO] Finished at: 2026-10-06T22:26:15-03:00
[INFO] -----
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM: ~/JAVA/java-preentrega$
```

Figura 10: Resultado de las pruebas unitarias

7. Conclusiones

La resolución de esta preentrega mediante la consola de Linux consolida un backend sólido y resiliente. El uso de validaciones defensivas blindadas contra el desbordamiento de búfer del `Scanner` asegura una experiencia estable para el usuario. El éxito en la ejecución de la suite de JUnit 5 valida matemáticamente el comportamiento del CRUD y el control de errores, dejando el ecosistema de TechLab con bases arquitectónicas profesionales y listo para su futura integración con sistemas de bases de datos relacionales persistentes.